Robot Devices, Kinematics, Dynamic and Control (EN.530.646)
Final Project

Vishek Masalia, Ruzie Geng, Pattharaphol Chainiwattana, Kaicheng Sun

1. Introduction

This project implements a push-and-place task using a UR5e robotic manipulator. The robot is taught a start pose via freedrive mode and autonomously pushes a foam cube along a straight line on a planar surface, followed by a return push to reposition the cube near its initial location. Two Cartesian-space control strategies are implemented and compared: inverse-kinematics–based control and resolved-rate (RR) control. The task is developed in both the ROS environment, supporting IK and RR control, and the MATLAB RTDE environment, implementing RR control for direct robot execution. Robot motion is planned in SE(3) with a fixed world-frame pushing direction, and safety constraints such as joint limits and collision avoidance are enforced.

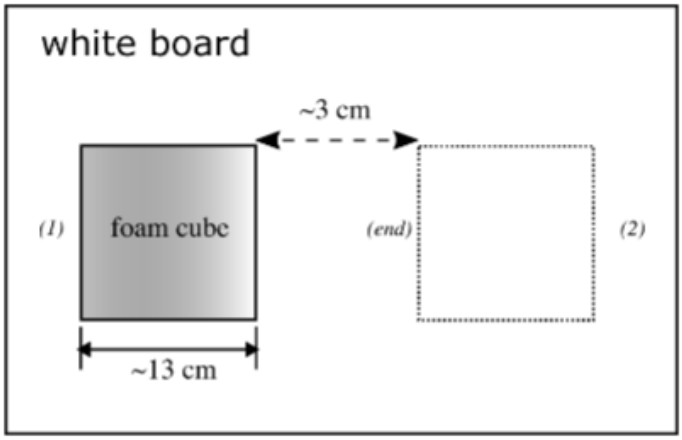


Fig. 1. Schematic of push-and-place task.

2. Problem Description and Objectives

The task requires the UR5e robot to push a foam cube approximately 16 cm along a straight line from a taught start pose, as shown in Fig. 1, and then automatically execute a return push without manually specifying a second target. The objectives are to achieve accurate world-frame Cartesian motion, evaluate different Cartesian control strategies, and ensure safe and robust execution on a real robotic system.

3. System Setup

3.1 Hardware

The hardware setup consists of a UR5e robotic manipulator operating on a whiteboard surface, a foam cube used as the object for the push-and-place task (Fig. 2), and a picking gripper mounted at the robot tool flange, which serves as the end-effector for pushing (Fig. 3).

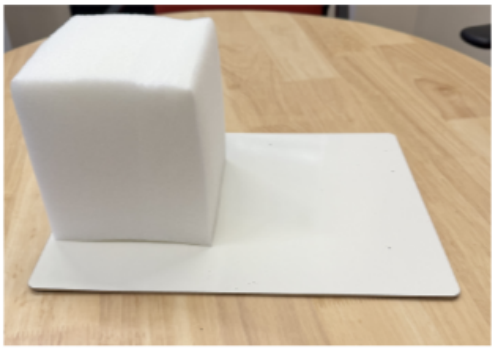


Fig. 2. Foam Cube on Whiteboard

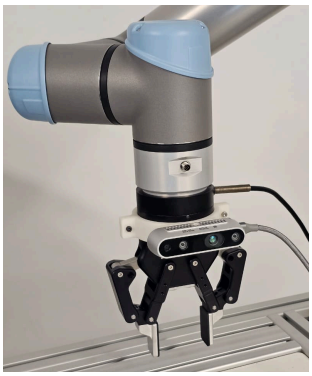


Fig. 3. Picking gripper used to push cube

3.2 Software Environments

The project is implemented and tested in the following software environments:

- ROS environment, used to implement both Resolved-Rate (RR) control and Inverse Kinematics (IK) control
    - Run ur_project.m
        - For RR control in ROS to be faster, optionally set variable T = 0.05 instead of 0.1 in file “urRRcontrol.m”. Change to 0.1 for RTDE.
    - For extra credit task, run ur_project_extra.m
- MATLAB RTDE environment, used to implement Resolved-Rate (RR) control for direct robot execution
    - Run ur_project_rtde.m

4. Coordinate Frames and Transformations

For this project, all motions are defined relative to the UR5e robot’s built-in frames (see Fig. 4):

- **Base Link Frame (‘base_link’)**: This is the fixed frame located at the base of the UR5e. All positions and transformations in the workspace are referenced with respect to this frame.
- **Tool Frame (‘tool0’)**: This frame is located at the UR5e tool flange. All Cartesian motions, including the push-and-place trajectories, are executed relative to this frame. The picking gripper is mounted on this flange, but no separate frame is defined for the gripper; its offset is handled conceptually in the trajectory planning.

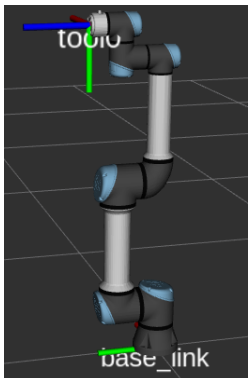


Fig. 4. Robot frames used in RViz

5. Control Algorithms

5.1 Resolved-Rate Control (RR)

In this section, resolved-rate (RR) control approach developed in Lab 3 is implemented to compute the next joint configuration according to

$$q_{i+1} = q_i - \Delta t K \left(J_{st}^{b}(q_i) \right)^{-1} \xi_i$$

where the control time step is Δt = 0.01s and the control gain is K = 1.5

Although the nominal control time step is 0.01 s, the actual time used to command joint motion and pause program execution is larger in order to ensure that the robot does not exceed its joint speed limits. In the ROS environment, a pause duration of 0.05 s is applied between control updates. In contrast, the RTDE environment requires a longer pause of 0.1 s, indicating a higher communication and execution latency compared to ROS.

5.2 Inverse Kinematics (IK) Control (ROS Only)

As discussed in class and implemented in the provided urInvKin.m function, for a given desired homogeneous transformation

$$g_d \in SE(3)$$

the inverse kinematics solver computes up to eight possible joint-space solutions for the UR5(e) robot. These solutions arise from the closed-form subproblem decomposition used in the solver. While all solutions satisfy the desired end-effector pose kinematically, not all are suitable for safe or smooth execution. Therefore, an appropriate solution must be selected from this set.

The goal of the inverse kinematics control strategy in this project is to select a joint configuration that achieves the desired pose while maintaining motion continuity, avoiding large joint jumps, and remaining close to the robot's current configuration.

Algorithm: IK Solution Selection

The following procedure is used to determine the optimal joint-space solution for each desired Cartesian pose along the push-and-place trajectory:

1. **Compute IK Solutions**
   For a given desired pose gdg_dgd, compute the set of all inverse kinematics solutions

   $$Q = \{q_1, q_2, \ldots q_n\},\ n =\leq 8$$

   using the urInvKin.m function.

2. **Filter Feasible Solutions and select the Best Solution**
   For each candidate solution $q_i \in Q$, apply a joint-space proximity filter. A solution for pure translation is considered feasible only if it lies within a bounded neighborhood of the previously selected joint configuration q_{prev}:

   $$q_{prev} - \Delta q \leq q_i \leq q_{prev} + \Delta q$$

   where Δq is a predefined joint threshold, $\Delta q = \pi/4$ radians. This threshold is created by assuming that the robot translates less than 30 cm from the previous position; the strict threshold allows only very similar joint-space to the previous one which is unique and has only one. Solutions that violate this condition are discarded to prevent large discontinuities in joint motion.

   $$q_{best} = q_{filter}$$

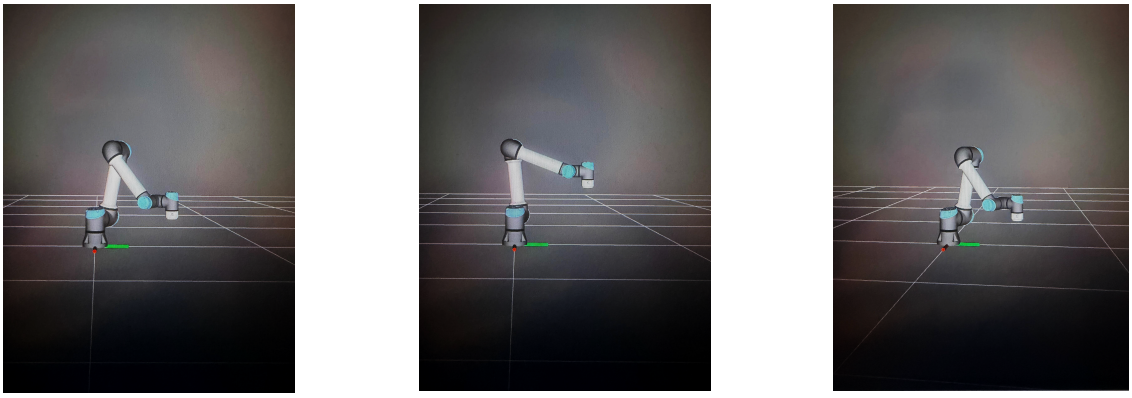
For the extra task, the solution corresponding to the general transformation (rotation and translation) is chosen by selecting q_i that minimizes the 2-norm of the difference between $q_{prev}$and q_i.

$$q_{best} = min\left(\left|\left|q_{prev} - q_i\right|\right|_2\right)$$

Each joint configuration is commanded using a fixed segment duration (tseg = 20 s), allowing the robot to reach the target smoothly. After each motion, the actual joint angles are read back from the robot and compared against the commanded values to evaluate execution accuracy.

This approach enables reliable execution of the push-and-place task while maintaining smooth joint motion and avoiding kinematic discontinuities.

6. Simulation Results



(a) (b) (c)

Fig. 5. Simulation snapshots from RViz (a) Taught position (b) Push forward complete (c) Push back complete.

6.1 Purpose and Setup

The simulation phase served as a critical verification step to validate the kinematic models and control algorithms prior to physical execution on the UR5e. Specifically, the correctness of the forward kinematics (urFwdKin.m), analytical inverse kinematics (urInvKin.m), and resolved-rate controller (urRRcontrol.m) were evaluated in a risk-free environment. The trajectory was defined by a sequence of target frames (gst1 through gst6) representing the taught start pose, the forward straight-line push, the lift,

the return push, and the retreat. The primary goal was to confirm that the end-effector followed a strictly linear path in the world frame while maintaining a constant orientation to ensure stable contact with the object.

6.2 Quantitative Error Analysis

To quantify the tracking accuracy, position (dR3) and orientation (dSO3) errors were calculated at each waypoint using the (generate_error_report.m) script. The results indicated a significant performance distinction between the two control methods. The Inverse Kinematics (IK) control, implemented in run_IK_based.m, achieved near-perfect accuracy.

Table 1: Errors between the desired location and the actual location

Position	**ROS (IK)**		**ROS (RR)**		**RTDE (RR)**	
	d_SO3	d_R3 (mm)	d_SO3	d_R3 (mm)	d_SO3	d_R3 (mm)
1 (Taught)	0.0000353	0.001507	0.014079	5.462	0.013962	5.259
2	0.0000887	0.001862	0.002984	9.924	0.000836	9.970
3	0.0000358	0.002115	0.10967	9.968	0.000041	9.861
4	0.0000627	0.001944	0.003816	9.938	0.000004	10.000
5	0.0000108	0.001485	0.005432	9.968	0.000000	9.876
6	0.0000196	0.00712	0.002669	9.888	0.000006	9.967

7. Experimental Results (Real Robot)

- Robot initialization
    - ROS environment
        - Run ros2 launch rdkdc ur5e_interface.launch.py in the terminal.
        - Run ur_project.m
    - RTDE environment
        - Run RTDE file in pendant
        - Run ur_project_rtde.m
- Execution steps

  The robotic arm is first taught a start position, then moves joint 3 by $-\pi/4$ relative to this position to reach a designated home position. It then returns to the teaching position using the control law. From there, the arm pushes the small cube along the x-axis to its target position. It then moves vertically upward along the z-axis. Once elevated beyond the cube's height, the arm rotates its gripper to face the opposite direction and lowers it until it grips the cube securely. Finally, it

pushes the cube back to its starting point along the opposite direction of the initial movement. Refer to Fig. 6.

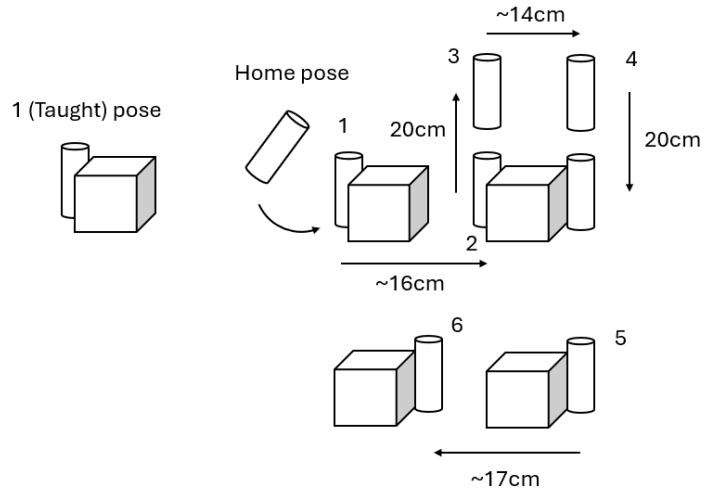


Fig. 6. Entire motion of robot in real life.

8. Discussion

RR Control vs. IK-Based Control

Both RR and IK-based control achieved comparable positional accuracy, with no observable end-effector errors at the target poses. This indicates that, for the relatively structured task and workspace considered, both approaches were sufficiently precise for reliable execution.

However, differences were observed in motion quality and convergence behavior. IK-based control produced noticeably smoother joint motion, while RR control exhibited mild but visible jerking during transitions between successive poses. Although the motion was not unstable or jittery, the discrete nature of the RR updates and the dependence on step-wise convergence contributed to these small discontinuities.

In terms of speed, IK-based control reached target poses faster and maintained a consistent velocity profile throughout the motion. RR control, by contrast, slowed down significantly as the end effector approached each target. This behavior can be attributed to the deliberately low linear and angular velocity thresholds used in the RR controller. These thresholds improved positional precision near the goal but introduced a trade-off in the form of reduced convergence speed. Additionally, RR control required explicit tuning of the control gain and step timing. During testing, higher gains led to rough and jerky motion, while lower gains improved smoothness but

reduced responsiveness. A gain of 1.5, combined with manual adjustment of the time step, resulted in acceptable performance without convergence failures.

Overall, IK-based control was easier to deploy and yielded smoother and faster motion with minimal tuning, whereas RR control offered fine-grained control over motion behavior at the cost of increased parameter sensitivity and tuning effort.

ROS vs. MATLAB RTDE Execution

Clear differences were also observed between ROS-based execution and MATLAB RTDE communication with the real robot. ROS proved more reliable and responsive, exhibiting lower communication latency and faster debugging cycles. The availability of simulation prior to real-world execution allowed errors to be identified early, significantly improving development efficiency and confidence before deployment.

In contrast, MATLAB RTDE introduced noticeable communication delays, requiring increased pause durations and longer joint motion times to ensure stable execution. Since the ROS and RTDE control logic was otherwise identical, diagnosing these issues was initially challenging. Ultimately, the limitations were traced to timing and synchronization constraints inherent in RTDE communication, which necessitated conservative timing choices and slowed down overall task execution.

Despite this, MATLAB RTDE was effective for rapid prototyping. Direct execution on the real robot enabled quick visual verification of motion behavior and facilitated iterative refinement without the overhead of maintaining a simulation environment. For final system development, however, ROS offered superior scalability, debuggability, and reliability due to its integrated simulation, visualization, and communication infrastructure.

Limitations and Future Improvements

A key limitation of the current RR implementation is the reliance on manually tuned pause durations and step times. This led to non-uniform motion behavior, with larger motions appearing jerky and smaller motions near the target becoming excessively slow. Incorporating adaptive step sizing, velocity scaling, or time-optimal trajectory generation could significantly improve smoothness and efficiency. Automatically computing pause durations based on joint velocity limits or distance-to-goal metrics would further reduce manual tuning and improve controller robustness.

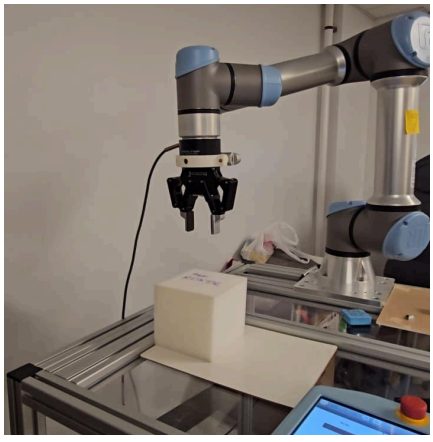
9. Extra Credit Task

As an additional task, we evaluated the positioning precision of the UR5e robotic manipulator using inverse kinematics (IK) control. The same foam cube used in the main push-and-place task was employed as a reference object to assess the robot's ability to repeatedly reach and make light contact with known surfaces.

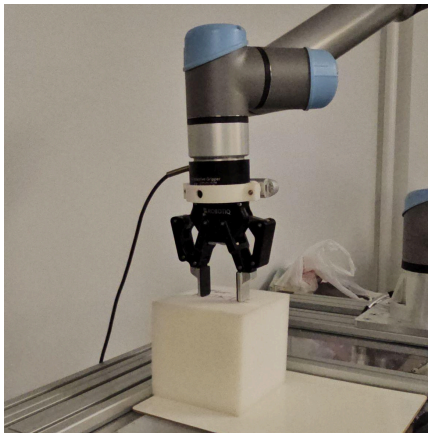
In this experiment, a reference pose corresponding to the top surface of the foam cube was first taught using manual positioning. This pose represents a configuration where the end effector is just barely touching the top surface of the cube (see Fig. 7(b)). After teaching this pose, the robot was commanded to move to a predefined home position located approximately 10 cm above the top surface of the cube (see Fig. 7(a)). From this home position, the robot autonomously returned to the taught pose using IK-based control, achieving near-contact with the cube surface without visibly compressing it.

Following this, the robot was commanded to move laterally to an adjacent side of the cube and once again approach the surface until light contact was achieved (see Fig. 7(d)). This sequence required precise Cartesian positioning and consistent inverse kinematics solution selection to avoid overshoot or unintended contact forces.

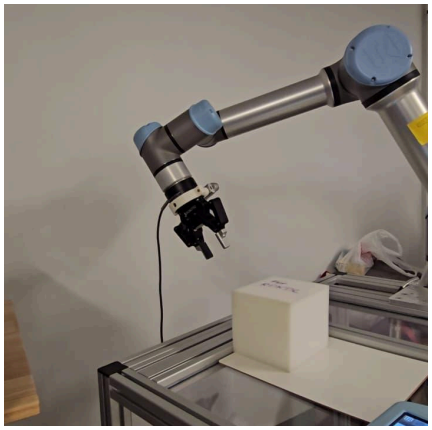
The robot successfully executed all stages of this task, demonstrating high positional accuracy and repeatability using IK control. The ability to approach multiple surfaces of the cube with minimal contact error highlights the precision of the UR5e and the robustness of the IK-based control strategy implemented in this project.



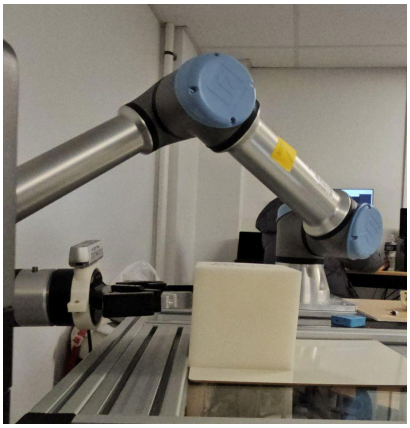
(a)



(b)



(c)



(d)

Fig. 7. Precision evaluation task using IK control: (a) home position above the foam cube, (b) manually taught near-contact pose at the top surface, (c) intermediate robot motion during surface transition, and (d) final target pose at an adjacent cube surface.

10. Conclusion

This project successfully implemented a fully autonomous push-and-place task using a UR5e manipulator, achieving reliable straight-line Cartesian motion in both simulation and real-robot execution. IK control produced smoother and faster motion with minimal tuning, while RR control offered precise, fine-grained behavior at the cost of slower convergence and manual tuning. ROS execution proved more reliable and responsive than MATLAB RTDE, though RTDE was effective for rapid prototyping. Overall, careful control strategy selection, environment choice, and tuning are critical for safe and efficient real-world robotic manipulation.

11. Workload Distribution

Team Member	**Contribution**
Vishek Masalia	Main task: RR control and IK control
Pattharaphol Chainiwattana	Main Task: RR control and IK control
Ruzie Geng	Extra credit task, Main Task: RTDE
Kaichen Sun	Main Task: Simulations, RTDE